مستند آموزشی Storybook و Dockerize کردن آن

1. مقدمه

در پروژه‌های رابط کاربری، تعداد کامپوننت‌ها به‌مرور افزایش پیدا می‌کند و مدیریت، تست و مستندسازی آن‌ها اهمیت زیادی پیدا می‌کند. یکی از ابزارهای کاربردی برای این منظور **Storybook**است.

Storybookمحیطی مستقل برای توسعه، نمایش، تست و مستندسازی کامپوننت‌های رابط کاربری فراهم می‌کند. در این محیط می‌توان هر کامپوننت را جدا از صفحات اصلی برنامه اجرا کرد و حالت‌های مختلف آن را بررسی نمود.

از طرف دیگر، با استفاده از **Docker**می‌توان نسخه ساخته‌شده Storybook را در یک محیط مستقل و قابل انتقال اجرا و Deploy کرد.

فرآیند کلی به شکل زیر است:

Development
↓
Create Components
↓
Create Stories
↓
Run Storybook
↓
Test & Document Components
↓
Build Storybook
↓
Docker Image
↓
Docker Container
↓
Web Server

Storybookدر حالت Production به یک **Static Web Application**تبدیل می‌شود که می‌تواند توسط یک Web Serverارائه شود(Storybook) .

---

2. Storybookچیست؟

Storybookیک ابزار Open Source برای توسعه و مستندسازی UI Components است.

به‌جای اینکه برای مشاهده یک کامپوننت مجبور باشیم کل Application را اجرا کنیم، می‌توانیم همان کامپوننت را به‌صورت مستقل در Storybook مشاهده کنیم.

برای مثال فرض کنیم یک Button داریم:

```
<Button variant="contained">
  ثبت
</Button>
```

در Storybook می‌توان حالت‌های مختلف آن را تعریف کرد:

```
Button
├── Default
├── Primary
├── Secondary
├── Disabled
├── Loading
└── Different Sizes
```

در نتیجه تیم توسعه می‌تواند رفتار و ظاهر کامپوننت‌ها را قبل از استفاده در صفحات اصلی بررسی کند.

---

3. مزایای استفاده ازStorybook

مهم‌ترین مزایای Storybook عبارت‌اند از:

3.1توسعه مستقل کامپوننت

هر کامپوننت را می‌توان بدون اجرای کامل Application توسعه داد.

3.2مستندسازی

Storyها می‌توانند به‌عنوان مستند تعاملی کامپوننت‌ها استفاده شوند.

3.3بررسی حالت‌های مختلف

حالت‌هایی مانند:

- Default
- Disabled
- Loading
- Error

- Empty
- Selected
- Hover
- Different Sizes

را می‌توان به‌صورت جداگانه نمایش داد.

3.4تست UI

Storybookامکان بررسی رفتار و ظاهر کامپوننت‌ها را در شرایط مختلف فراهم می‌کند.

3.5همکاری بین اعضای تیم

طراح، Frontend Developer، QAو سایر اعضای تیم می‌توانند UI Components را بدون نیاز به اجرای کل سیستم بررسی کنند.

3.6ایجاد Design System

Storybookمی‌تواند به‌عنوان یک محیط مرکزی برای نمایش و مستندسازی Design System مورد استفاده قرار گیرد.

---

4. ساختار کلیStorybook

پس از نصب Storybook معمولاً یک پوشه با نام زیر ایجاد می‌شود:

.storybook/

این پوشه محل قرارگیری تنظیمات Storybook است.

ساختار ساده یک پروژه می‌تواند به شکل زیر باشد:

```
project/
│
├── src/
│   ├── components/
│   │   ├── Button/
│   │   │   ├── Button.jsx
│   │   │   └── Button.stories.jsx
│   │   │
│   │   └── Input/
│   │       ├── Input.jsx
```

```
│   │       └── Input.stories.jsx
│   │
│   └── ...
│
├── .storybook/
│   ├── main.js
│   └── preview.js
│
├── package.json
└── ...
```

5.نصبStorybook

Storybookدارای CLI است و نصب آن می‌تواند از طریق Setup Wizard انجام شود. مستندات رسمی فعلی Storybookاستفاده از دستور زیر را برای راه‌اندازی پیشنهاد می‌کنند:

```
npm create storybook@latest
```

این فرآیند تنظیمات موردنیاز را بر اساس Framework و ساختار پروژه تشخیص می‌دهد(Storybook) .

در پروژه‌های مبتنی بر pnpm نیز می‌توان از دستور متناسب با Package Manager استفاده کرد.

پس از نصب، معمولاً دستورات زیر در package.jsonقرار می‌گیرند:

```
{
  "scripts": {
    "storybook": "storybook dev -p 6006",
    "build-storybook": "storybook build"
  }
}
```

6.اجرای Storybook در محیطDevelopment

برای اجرایStorybook:

```
npm run storybook
```

یا:

```
pnpm storybook
```

Storybookیک Development Server اجرا می‌کند و امکان مشاهدهStory ها و تغییرات آن‌ها را به‌صورت لحظه‌ای فراهم می‌کند(Storybook) .

در این حالت، هدف اصلی **Development**است.

بنابراین Development Storybook را نباید با نسخه Production یکی در نظر گرفت.

---

7. مفهومStory

اصلی‌ترین مفهوم درStorybook ، **Story**است.

هر Story یک حالت مشخص از یک Component را تعریف می‌کند.

برای مثال:

```
import Button from "./Button";

export default {
  title: "Components/Button",
  component: Button,
};

export const Primary = {
  args: {
    children: "ثبت",
    variant: "contained",
  },
};

export const Secondary = {
  args: {
    children: "انصراف",
    variant: "outlined",
  },
};
```

در نتیجه Storybook دو حالت برای Button نمایش می‌دهد:

```
Button
├── Primary
└── Secondary
```

---

8. استفاده ازArgs

یکی از قابلیت‌های مهمStorybook ، استفاده از argsاست.

به کمک argsمی‌توان Props مختلف یک Component را کنترل کرد.

برای مثال:

```
export const Primary = {
  args: {
    children: "ثبت اطلاعات",
    disabled: false,
  },
};
```

یا:

```
export const Disabled = {
  args: {
    children: "ثبت اطلاعات",
    disabled: true,
  },
};
```

این روش باعث می‌شود حالت‌های مختلف Component بدون تغییر مستقیم کد اصلی قابل بررسی باشند.

9.Controls

Storybookمی‌تواند Props کامپوننت را در محیط UI قابل تغییر کند.

برای مثال:

Button

Label:
[ثبت اطلاعات]

Variant:
[contained ▼]

Disabled:
[false]

Size:
[medium ▼]

در نتیجه Developer یا Tester می‌تواند بدون تغییر کد، رفتار Component را بررسی کند.

---

10. تنظیمات اصلیStorybook

دو فایل مهم در تنظیمات Storybook معمولاً عبارت‌اند از:

.storybook/main.js
.storybook/preview.js

main.js

این فایل برای تنظیمات اصلی Storybook استفاده می‌شود.

نمونه:

```
const config = {
  stories: [
    "../src/**/*.stories.@(js|jsx|ts|tsx)",
    "../src/**/*.mdx",
  ],

  addons: [
    "@storybook/addon-essentials",
  ],

  framework: {
    name: "@storybook/react-vite",
    options: {},
  },
};

export default config;
```

Storybookبسته به Framework و Builder مورد استفاده می‌تواند با Vite یا Webpack اجرا شود .
(Storybook)

---

11. تنظیماتPreview

فایل:

.storybook/preview.js

برای تنظیمات عمومیStory ها استفاده می‌شود.

برای مثال اگر پروژه RTL باشد، می‌توان تنظیمات مربوط به Direction را در این قسمت یا در Theme/Decoratorمربوط به UI اعمال کرد.

نمونه:

```
const preview = {
  parameters: {
    layout: "centered",
  },
};

export default preview;
```

12.پشتیبانی از RTL وLTR

در پروژه‌هایی که از زبان‌هایی مانند فارسی و عربی استفاده می‌کنند، توجه به **RTL**اهمیت زیادی دارد.

در مقابل، زبان‌هایی مانند انگلیسی معمولاً در حالت **LTR**نمایش داده می‌شوند.

بنابراین بهتر است Storybook به شکلی تنظیم شود که بتوان هر دو حالت را بررسی کرد.

RTL

```
<div dir="rtl">
  <Button>ثبت اطلاعات</Button>
</div>
```

LTR

```
<div dir="ltr">
  <Button>Submit</Button>
</div>
```

در طراحی Component باید موارد زیر نیز بررسی شوند:

- Alignment
- Margin
- Padding
- Icon Position
- Text Direction
- Input Direction
- Table Direction
- Menu Direction
- Tooltip Position
- Pagination

---

13. Theme در Storybook

اگر پروژه از یک Design System استفاده می‌کند، بهتر است Theme اصلی نیز در Storybook اعمال شود.

برای مثال:

```
import { ThemeProvider } from "@mui/material/styles";
import theme from "../src/theme";

export const decorators = [
  (Story) => (
    <ThemeProvider theme={theme}>
      <Story />
    </ThemeProvider>
  ),
];
```

به این ترتیب Component در Storybook با همان Theme اصلی پروژه نمایش داده می‌شود.

این موضوع برای مواردی مانند:

- Typography
- Colors
- Spacing
- Border Radius
- Shadows
- RTL

- Component Overrides

اهمیت زیادی دارد.

---

14. ساخت نسخهProduction

بعد از پایان Development باید Storybook به نسخه Production تبدیل شود.

دستور:

```
npm run build-storybook
```

یا:

```
pnpm build-storybook
```

Storybookدر این مرحله یک نسخه Static تولید می‌کند که معمولاً در پوشه‌ای مانند زیر قرار می‌گیرد:

```
storybook-static/
```

این خروجی شامل فایل‌هایHTML ، JavaScript، CSSو سایرAsset های موردنیاز است و می‌تواند توسط Web Serverارائه شود([Storybook]) .

---

15. تفاوت Development وProduction

مورد	Development	Production
هدف	توسعه	انتشار
اجرا	storybook dev	storybook build
تغییرات لحظه‌ای	دارد	ندارد
مناسب برای	Developer Server/Deployment	
خروجیStatic	خیر	بله
نیاز بهBuild	خیر	بله

بنابراین در Docker Production بهتر است خود Development Server را اجرا نکنیم.

به‌جای آن:

```
Source Code
    ↓
Storybook Build
    ↓
Static Files
    ↓
Web Server
```

---

16. Dockerچیست؟

Dockerفناوری‌ای برای بسته‌بندی و اجرای Application در محیط‌های ایزوله به نام **Container**است.

در این روش Application به همراه نیازمندی‌های لازم در قالب یک Image قرار می‌گیرد.

ساختار کلی:

```
Dockerfile
    ↓
Docker Image
    ↓
Docker Container
    ↓
Application
```

مزیت اصلی این روش این است که محیط اجرای Application قابل تکرار و قابل انتقال می‌شود.

---

17. چرا Storybook را Dockerize کنیم؟

پس از Build شدنStorybook ، خروجی آن یک Static Web Application است.

بنابراین می‌توان آن را داخل یک Container قرار داد و با یک Web Server مانند Nginx ارائه کرد.

مزایای این روش:

- اجرای یکسان در محیط‌های مختلف
- ساده‌تر شدنDeployment
- جداسازی محیط Build وRuntime

- امکان استفاده درCI/CD
- کاهش وابستگی به Node.js درRuntime
- قابل انتقال بودنDocker Image

Storybookنیز برای انتشار، Buildشدن به یک Static Web Application را پشتیبانی می‌کند(Storybook) .

---

18. Dockerfile چیست؟

Dockerfileیک فایل متنی است که مراحل ساخت Docker Image را مشخص می‌کند.

برای Storybook می‌توان از یک **Multi-Stage Build**استفاده کرد.

ساختار کلی:

```
Stage 1
Build Storybook
   ↓
Stage 2
Web Server
   ↓
Static Storybook
```

> استفاده از Multi-Stage Build باعث می‌شود ابزارهای موردنیاز برای Build در Image نهایی قرار نگیرند و Imageنهایی سبک‌تر شود .Docker نیز این روش را برای جدا کردن Build Environment از Runtime Environmentتوصیه می‌کند(Docker Documentation) .

---

19. Dockerfileنمونه

یک نمونه عمومی:

```
# Stage 1: Build
FROM node:20-alpine AS builder

WORKDIR /app

RUN corepack enable

COPY package.json pnpm-lock.yaml ./
```

```
RUN pnpm install --frozen-lockfile

COPY . .

RUN pnpm build-storybook


# Stage 2: Runtime
FROM nginx:alpine

COPY --from=builder /app/storybook-static /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

در این ساختار:

Stageاول

```
FROM node:20-alpine AS builder
```

محیط Node.js برای نصب Dependencies و Build کردن Storybook ایجاد می‌شود.

سپس:

```
RUN pnpm install --frozen-lockfile
```

Dependenciesنصب می‌شوند.

و در نهایت:

```
RUN pnpm build-storybook
```

نسخه Production ساخته می‌شود.

Stageدوم

```
FROM nginx:alpine
```

یک Web Server سبک برای ارائه فایل‌های Static ایجاد می‌شود.

سپس:

```
COPY --from=builder /app/storybook-static /usr/share/nginx/html
```

فقط خروجی نهایی Storybook به Image نهایی منتقل می‌شود.

در نتیجه Node.js و Dependencies مربوط به Build در Runtime نهایی وجود ندارند.

---

20. فایل.dockerignore

برای جلوگیری از انتقال فایل‌های غیرضروری به Docker Build Context ، بهتر است فایل .dockerignore ایجاد شود.

نمونه:

```
node_modules
storybook-static
dist
.git
.gitignore
Dockerfile
*.log
coverage
.env
```

این کار باعث کاهش حجم Build Context و جلوگیری از انتقال فایل‌های غیرضروری می‌شود.

Docker نیز استفاده از .dockerignore را برای حذف فایل‌هایی مانند node_modules، فایل‌های Build و فایل‌های محیطی پیشنهاد می‌کند(Docker Documentation) .

---

21. ساخت Docker Image

بعد از ایجاد Dockerfile:

```
docker build -t storybook-app .
```

در این مرحله Docker:

```
Dockerfile
   ↓
Build Stage
   ↓
Install Dependencies
```

↓
Build Storybook
↓
Runtime Stage
↓
Docker Image

را ایجاد می‌کند.

---

22. اجرایContainer

بعد از ساخته شدن:Image

```
docker run -d -p 8080:80 --name storybook-container storybook-app
```

توضیح:

```
8080:80
```

یعنی:

```
Host Port 8080
    ↓
Container Port 80
```

در نتیجه Web Server داخل Container روی Port 80 اجرا می‌شود و از طریق Port 8080 روی سیستم Hostقابل دسترسی است.

---

23. مشاهدهContainer ها

برای مشاهدهContainer های در حال اجرا:

```
docker ps
```

برای مشاهده تمامContainer ها:

```
docker ps -a
```

---

24. مشاهدهLog ها

برای مشاهدهLog های:Container

```
docker logs storybook-container
```

برای مشاهدهLog ها بهصورت زنده:

```
docker logs -f storybook-container
```

25.توقفContainer

```
docker stop storybook-container
```

و برای حذف:

```
docker rm storybook-container
```

---

26. حذفDocker Image

برای مشاهدهImage ها:

```
docker images
```

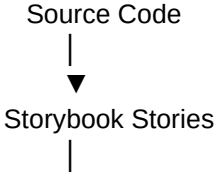
و برای حذف:

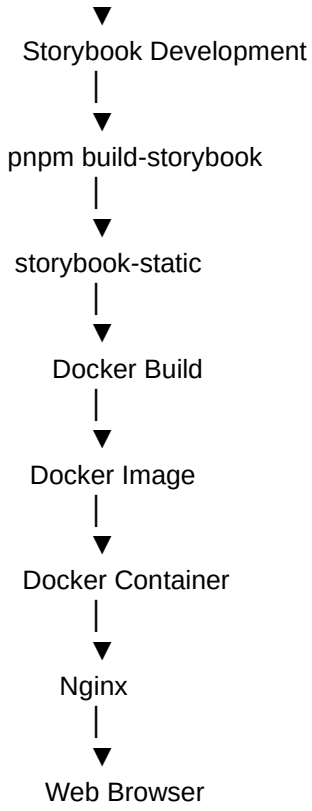
```
docker rmi storybook-app
```

---

27. فرآیند کامل Dockerize کردن

فرآیند نهایی را می‌توان به شکل زیر خلاصه کرد:



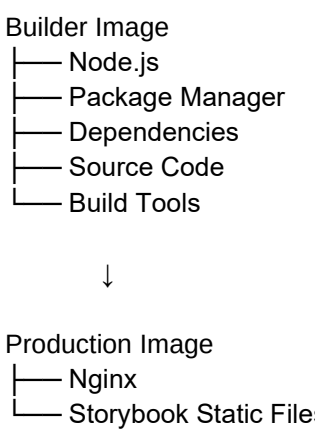


---

28. چرا Multi-Stage Build ؟

در روش ساده ممکن است Node.js ، Package Manager، Dependencies و ابزارهای Build همگی در Image نهایی باقی بمانند.

اما در Multi-Stage Build:



بنابراین Image نهایی فقط چیزهایی را دارد که برای اجرای Production لازم هستند.

Dockerمستند کرده است که Multi-Stage Build می‌تواند حجم Image نهایی و سطح حمله را کاهش دهد .
(Docker Documentation)

---

29. نکات مهم در Dockerize کردنStorybook

نکته اول Build :را داخل Docker انجام دهید

بهتر است فرآیند Build در محیط کنترل‌شده Docker انجام شود:

```
RUN pnpm build-storybook
```

این کار باعث می‌شود Build Environment قابل تکرارتر باشد.

نکته دوم: از --frozen-lockfile استفاده کنید

در پروژه‌هایی که Lock File دارند:

```
pnpm install --frozen-lockfile
```

باعث می‌شودDependency Version ها مطابق Lock File نصب شوند.

نکته سوم node_modules :را وارد Docker Context نکنید

در.dockerignore:

```
node_modules
```

قرار دهید.

نکته چهارم: فایل‌های Environment را وارد Image نکنید

در صورت وجود اطلاعات حساس:

```
.env
```

را در .dockerignoreقرار دهید.

نکته پنجم Production Server :را از Development Server جدا کنید

در Production نیازی به اجرای:

storybook dev

نیست.

بهتر است:

storybook build
↓
Static Files
↓
Nginx

استفاده شود.

---

30. خطاهای رایج

خطای عدم تطابقNode.js

برخی نسخه‌های جدید Storybook به نسخه‌های جدید Node.js نیاز دارند. بنابراین قبل از نصب باید نسخه Node.jsو نیازمندی‌های نسخه Storybook بررسی شود. مستندات فعلیStorybook ، +20 Node.jsو ابزارهای مدرن مرتبط را در نیازمندی‌های نصب ذکر می‌کنند(Storybook) .

بررسی نسخه:

node -v

خطایDependency

در صورت وجود Lock File بهتر است:

pnpm install --frozen-lockfile

استفاده شود.

خطایBuild Storybook

برای بررسی جزئیات:Build

```
pnpm build-storybook
```

را خارج از Docker نیز اجرا کنید.

اگر Build در محیط اصلی موفق نباشد، Dockerنیز معمولاً نمی‌تواند آن را با موفقیت Build کند.

خطایPort

اگر Port موردنظر قبلاً استفاده شده باشد:

```
docker run -d -p 8081:80 storybook-app
```

می‌توان Port سمت Host را تغییر داد.

ساختار:

8081 → 80

---

31. پیشنهاد ساختار استاندارد

برای یک پروژهFrontend ، ساختار پیشنهادی:

```
project/
│
├── src/
│   ├── components/
│   ├── stories/
│   └── ...
│
├── .storybook/
│   ├── main.js
│   └── preview.js
│
├── public/
│
├── Dockerfile
├── .dockerignore
├── package.json
└── pnpm-lock.yaml
```

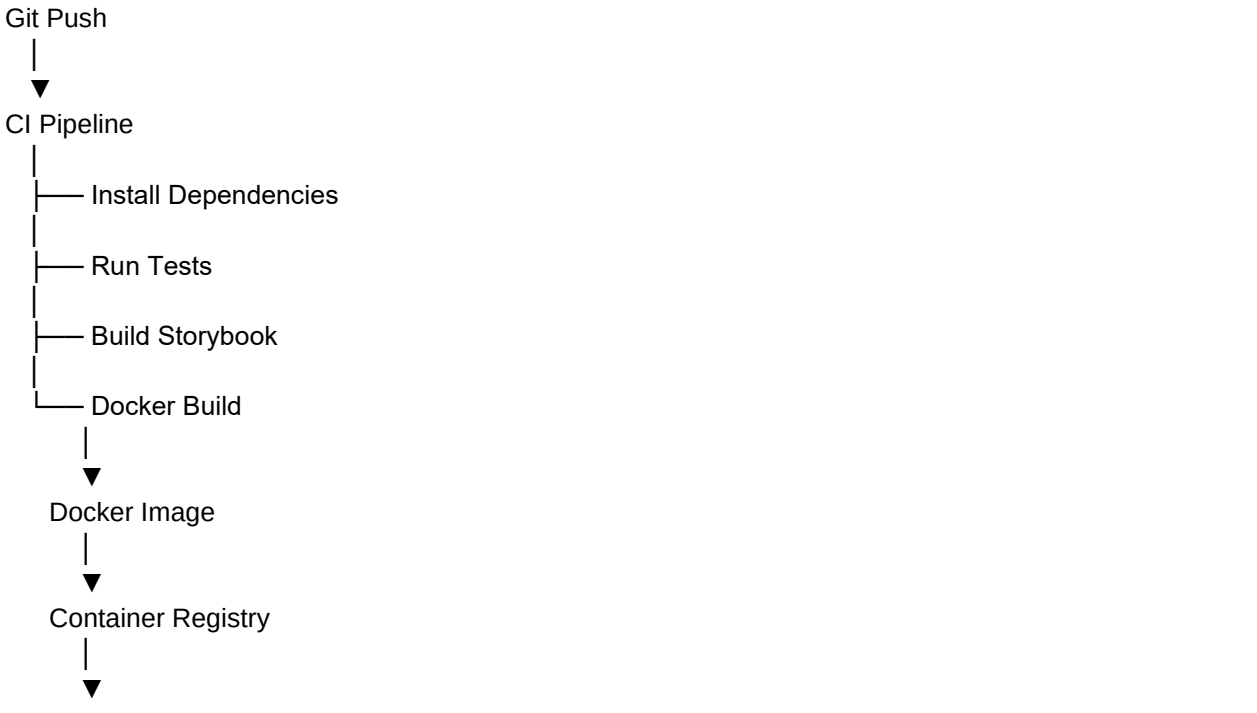
در این ساختار:

- src → کد اصلی
- .storybook → تنظیمات Storybook
- stories → Storyهای کامپوننت‌ها
- Dockerfile → ساخت Docker Image
- .dockerignore → حذف فایل‌های غیرضروری از Build Context
- package.json → Scripts و Dependencies
- pnpm-lock.yaml → نسخه دقیق Dependencies

---

32. فرآیند پیشنهادی برایCI/CD

در محیط سازمانی می‌توان فرآیند را به شکل زیر پیاده‌سازی کرد:



در این مدل، هر تغییر درComponent ها می‌تواند باعث ایجاد نسخه جدید Storybook شود.

---

33. جمع‌بندی

Storybookیک ابزار مناسب برای **توسعه مستقل، نمایش، تست و مستندسازی Component های UI** است.

در محیط Development می‌توان با اجرای:

```
pnpm storybook
```

کامپوننت‌ها را به‌صورت تعاملی بررسی کرد.

پس از تکمیل توسعه، با:

```
pnpm build-storybook
```

یک نسخه Static از Storybook ایجاد می‌شود. این خروجی می‌تواند توسط یک Web Server ارائه شود .
(Storybook)

برای Deployment نیز می‌توان از Docker استفاده کرد:

```
Storybook
   ↓
Static Build
   ↓
Docker Multi-Stage Build
   ↓
Nginx
   ↓
Docker Container
   ↓
Production
```

استفاده از **Multi-Stage Docker Build**باعث می‌شود محیط Build از محیط Runtime جدا شود و Image نهایی فقط شامل فایل‌های موردنیاز برای اجرای Production باشد(Docker Documentation) .

دستورات اصلی

```
# نصب و راه‌اندازی
npm create storybook@latest

# اجرای Development
pnpm storybook

# ساخت Production
pnpm build-storybook

# ساخت Docker Image
docker build -t storybook-app .
```

```
# اجرای Container
docker run -d -p 8080:80 --name storybook-container storybook-app

# مشاهده Container
docker ps

# مشاهده Log
docker logs storybook-container

# توقف
docker stop storybook-container
```

34.ورود به(Docker Registry (Docker Login

برای ارسال Docker Image به Registry ابتدا باید وارد Registry شویم:

```
docker login <registry-address>
```

پس از اجرای دستور، نام کاربری و رمز عبور درخواست می‌شود:

```
Username: <username>
Password: <password>
```

در صورت موفقیت، پیام زیر نمایش داده می‌شود:

```
Login Succeeded
```

بهتر است اطلاعات Username و Password مستقیماً داخل دستورات یا فایل‌های پروژه قرار نگیرند.

---

35. ساختDocker Image

ابتدا Image ساخته می‌شود:

```
docker build -t storybook-app:1.0.0 .
```

در این دستور:

```
storybook-app → نام Image
1.0.0         → نسخه
.             → مسیر Build Context
```

برای بررسی Image:

```
docker images
```

---

36. Tag کردن Image برایRegistry

برای Push کردنImage ، باید Image را با نام Registry و Repository موردنظر Tag کنیم.

ساختار کلی:

```
docker tag <image-name>:<version> <registry>/<repository>:<version>
```

مثال عمومی:

```
docker tag storybook-app:1.0.0 <registry-address>/storybook-app:1.0.0
```

بعد از Tag کردن، می‌توانImage ها را بررسی کرد:

```
docker images
```

مثلاً:

```
REPOSITORY                          TAG
storybook-app                       1.0.0
<registry-address>/storybook-app    1.0.0
```

---

37. استفاده ازTag های مختلف

برای مدیریت نسخه‌ها می‌توانTag های مختلف ایجاد کرد:

```
docker tag storybook-app:1.0.0 <registry-address>/storybook-app:1.0.0
docker tag storybook-app:1.0.0 <registry-address>/storybook-app:latest
```

در این حالت یک Image با دو Tag در Registry قابل شناسایی است:

```
1.0.0
latest
```

برای محیط‌های Production بهتر است از **Versionمشخص** استفاده شود:

```
1.0.0
1.0.1
1.1.0
```

زیرا مشخص است دقیقاً چه نسخه‌ای Deploy شده است.

---

38. Push کردنDocker Image

بعد از Login و Tag کردن، Imageرا به Registry ارسال می‌کنیم:

```
docker push <registry-address>/storybook-app:1.0.0
```

برای ارسال latest Tag نیز:

```
docker push <registry-address>/storybook-app:latest
```

در هنگامPush ، Dockerلایه‌های Image را به Registry منتقل می‌کند:

```
The push refers to repository
...
Layer already exists
Pushed
Digest: ...
```

در صورت موفقیت، Imageدر Registry قابل استفاده خواهد بود.

---

39. Pull کردنImage

برای دریافت Image از Registry روی یک سیستم دیگر:

```
docker pull <registry-address>/storybook-app:1.0.0
```

پس از دریافت:

```
docker images
```

می‌توان Image را مشاهده کرد.

---

40. اجرای Image دریافت‌شده

پس از Pull کردن:

```
docker run -d \
  -p 8080:80 \
  --name storybook-container \
  <registry-address>/storybook-app:1.0.0
```

در PowerShell ویندوز نیز می‌توان دستور را در یک خط نوشت:

```
docker run -d -p 8080:80 --name storybook-container <registry-address>/storybook-app:1.0.0
```

---

41. بررسیContainer

بعد از اجرا:

```
docker ps
```

برای مشاهده تمامContainer ها:

```
docker ps -a
```

---

42. دریافتLog

برای مشاهده Log مربوط به Container:

```
docker logs storybook-container
```

برای مشاهده لحظه‌ای:

```
docker logs -f storybook-container
```

برای نمایش آخرین 100 خط:

```
docker logs --tail 100 storybook-container
```

برای نمایش Log همراه با زمان:

```
docker logs -t storybook-container
```

ترکیب آن‌ها:

```
docker logs -t -f --tail 100 storybook-container
```

---

43. بررسی وضعیت و مصرف منابع

برای بررسی وضعیت Container:

```
docker inspect storybook-container
```

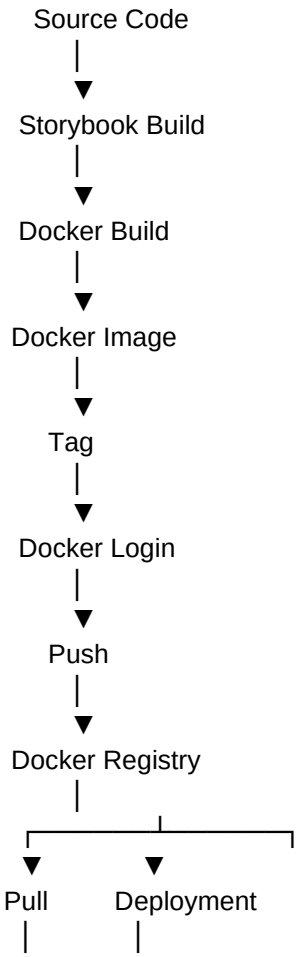
برای بررسی CPU و Memory:

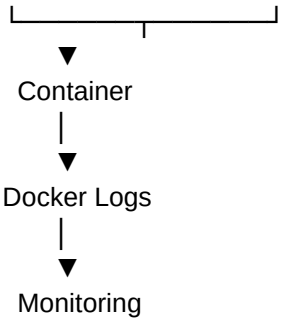
```
docker stats storybook-container
```

---

44. فرآیند کامل انتشار

فرآیند کامل Dockerize و انتشار Storybook به شکل زیر است:





---

45. خلاصه دستورات اصلی

```
# 1. Build
docker build -t storybook-app:1.0.0 .

# 2. Login
docker login <registry-address>

# 3. Tag
docker tag storybook-app:1.0.0 <registry-address>/storybook-app:1.0.0

# 4. Push
docker push <registry-address>/storybook-app:1.0.0

# 5. Pull
docker pull <registry-address>/storybook-app:1.0.0

# 6. Run
docker run -d -p 8080:80 --name storybook-container <registry-address>/storybook-app:1.0.0

# 7. Check Container
docker ps

# 8. View Logs
docker logs storybook-container

# 9. Live Logs
docker logs -f storybook-container

# 10. Resource Monitoring
docker stats storybook-container
```